Practice 2

Security Gate for a Release Candidate

SAST · DAST · secrets · SBOM · dependencies · container · 65 minutes

The task

Release candidate 2.0
a C application and a Docker image

→

Decision
RELEASE or BLOCK
backed by evidence

> A scanner produces a finding; responsibility for the decision stays with the team

65 minutes · eight stages

05 · Setup lab and repository	**05 · Build** test and the problem branch	**12 · SAST** the double `free` path and the SARIF report	**03 · DAST** ASAN build and an array overflow
10 · Secrets file and Git history	**12 · Supply chain** SBOM and CVE	**09 · Container** configuration and privileges	**09 · Fixes** code, bounds, secrets, image and the Gate

One ready-made Ubuntu environment

Host

```
./lab.sh start
./lab.sh shell
```

→

Ubuntu 24.04

Clang + ASAN · Git · Gitleaks

Syft · Grype · Trivy

`/work/repository` · `/work/reports`

> After logging in, all commands run in a single terminal

The green test trap

```
100% tests passed, 0 tests failed
session closed
```

> The functional test is green. Is that enough to allow the release?

Why Clang Static Analyzer

Free

open source and available locally

Semantic

tracks memory state across branches

Explainable

builds a trace from `malloc` to the second `free`



Checkpoint 1 · memory ownership path analysis

SARIF: the SAST report for machines

`runs → tool`

the analyzer `clang` and the rule list `unix.Malloc`

`results[]`

`ruleId` · `level` · `message` · `locations` with file and line

`codeFlows`

the trace: allocation, release, repeat; `importance` of a step

> One JSON format (OASIS, 2.1.0): reports of different analyzers go to one place

DAST: run the program under ASAN

Build
`clang -fsanitize=address`

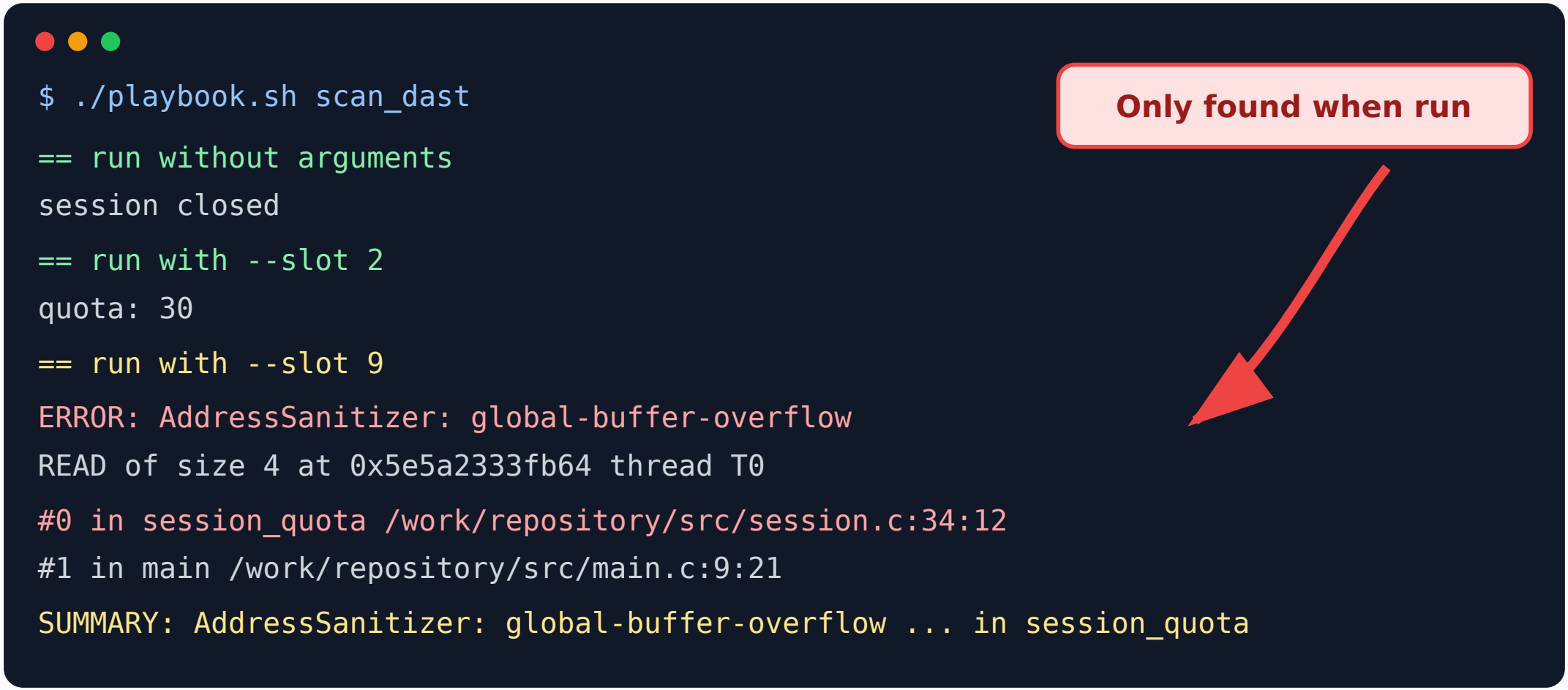
→

Three runs
no arguments
`--slot 2` · `--slot 9`

→

ASAN report
kind of error
function and line

ASAN checks memory while the program runs; LeakSanitizer prints leaks at exit



Checkpoint 2 · dynamic analysis sees only the paths that run

SAST and DAST see different things

	SAST	DAST
Reads	the source code, without running it	the running program
Found	double `free`, `session.c:27`	`quota[9]`, `session.c:34`
Missed	an index from the command line	`--reject`: that path was not run
Fix	one place to `free`	bounds check on `slot`

> Neither method replaces the other: SAST sees code that did not run, DAST sees real data

The ownership rule

Before
error branch: `free`
common exit: `free`

→

After
both branches jump
to a single `cleanup`

> `free(NULL)` hides the symptom; a single cleanup point fixes the model

```
$ ./playbook.sh scan_secrets
2 commits scanned.
leaks found: 2

RuleID: training-api-token
File: config/app.env
Commit: prepare release 2.0

RuleID: training-api-token
File: config/legacy.env
Commit: bootstrap release candidate
```



Checkpoint 3 · deleting the file is not enough

Deleting a file ≠ fixing the leak

Action	What it fixes	What remains
Delete the line	the current file	the token is still valid
Delete the file	the working tree	the commit is still accessible
Rewrite Git	repository history	the compromised token
Rotation	access with the old token	the new one must be delivered

> First revoke and replace the secret, then clean up and coordinate the history

SBOM and CVE are different artifacts

Syft → SBOM
name · version · PURL
contents of a specific image

→

Grype → findings
advisory · severity · fix
state of the CVE database

> Keep the SBOM with the release; re-check vulnerabilities against it later

```
$ ./playbook.sh make_sbom
✔ Loaded image practice-2-release:student
✔ Cataloged contents
SBOM: reports/sbom.cdx.json

$ ./playbook.sh scan_container
Dockerfile: Tests 21 · Failures 2 HIGH
AVD-DS-0002 · no non-root USER
AVD-DS-0029 · no --no-install-recommends

Decision: BLOCK
```



Checkpoint 4 · image contents and container configuration

Attack surface of the original image

Build = runtime

compiler and CMake ship with the product

Root

the process gets excess privileges

`privileged`

container isolation is weakened

Bind mount

source code is writable

Capabilities

not explicitly restricted

Filesystem

writable everywhere

Hardening reference

```
FROM debian:12-slim AS build
RUN apt-get install -y --no-install-recommends build-essential cmake
# build and tests

FROM debian:12-slim
RUN useradd --system --gid app --create-home user
COPY --from=build /src/build/release-agent ./release-agent
USER user
HEALTHCHECK --interval=30s --timeout=3s CMD ["/app/release-agent"]
ENTRYPOINT ["/app/release-agent"]
```

> Multi-stage removes the build toolchain from runtime and shrinks the SBOM

Security Gate

SAST	DAST	Secrets	SBOM	CVE	Config
0 blockers	0 ASAN errors	0 findings	generated	decision	0 HIGH

PASS opens the release

```
$ ./playbook.sh verify_release

No analyzer findings

3 commits scanned.

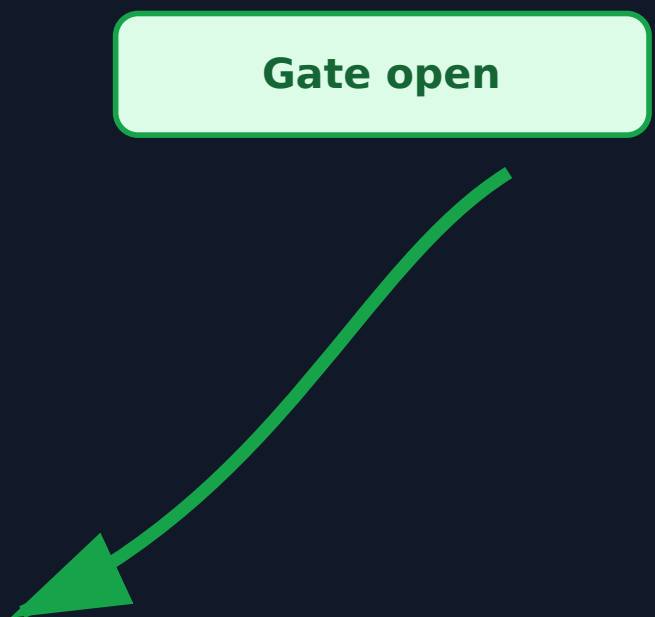
no leaks found

SBOM generated · CycloneDX

Trivy: 0 HIGH · 0 CRITICAL


[PASS] Security Gate: релиз-кандидат принят

[INFO] components in SBOM: 89
```



Checkpoint 5 · re-check after the fixes

Before and after

Control	Before	After	Reason
SAST	1 blocker	0	single cleanup
DAST	1 ASAN error	0	bounds check
Gitleaks	2	0	rotation and history
SBOM	≈234	≈89	multi-stage
Grype	3 Critical · 25 High	1 Critical · 8 High	fewer packages in runtime
Trivy	2 HIGH	0 HIGH	non-root and minimal packages

> CVE counts change; the contents, fixes and decision must be reproducible

Conclusion

A secure release is a provable chain of controls from source code to runtime configuration.

Find → explain → fix → re-check → decide